GOOGLE DATA ANALYTICS

TEST CASE 1: Retail Sales Performance Analysis

Aim:
Using Google Colab, perform an analysis on a dataset of product sales to answer the following.

Requirements:

- Google Colab
- Python 3.x
- Libraries: pandas, matplotlib

Algorithm:

1. Create a dataset of products with Quantity and Price.
2. Fill missing values in the *Price* column with the average price of the respective product.
3. Calculate **Total_Sales = Quantity × Price.**
4. Identify the product with the highest total sales.
5. Plot a bar chart showing total sales by product.

Procedure:

1. Open Google Colab and define the dataset.
2. Apply the algorithm step-by-step.
3. Execute and observe outputs and visualizations.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
   'Product': ['Laptop','Mobile','Tablet','Laptop','Mobile','Tablet'],
   'Quantity': [5,10,7,3,5,2],
   'Price': [50000,15000,20000,None,16000,21000]
}

df = pd.DataFrame(data)
df['Price'] = df.groupby('Product')['Price'].transform(lambda x: x.fillna(x.mean()))
```

```
df['Total_Sales'] = df['Quantity'] * df['Price']


top_product = df.groupby('Product')['Total_Sales'].sum().idxmax()
print(f"Product with highest total sales: {top_product}")


sales_summary = df.groupby('Product')['Total_Sales'].sum()
sales_summary.plot(kind='bar', color='skyblue', figsize=(8,5))
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
```

Output:

Product with highest total sales: Laptop

(Bar chart displayed in Colab)

Result:
The program successfully calculates total sales, identifies the top product, and visualizes total sales by product.

TEST CASE 2: City Temperature Monitoring

Aim:
Using Google Colab, perform an analysis on a dataset of city temperatures to answer the following.

Requirements:

- Google Colab
- Python 3.x
- Libraries: pandas, matplotlib

Algorithm:

1. Create a dataset with Min_Temp and Max_Temp for several days.
2. Fill missing values with column averages.
3. Compute **Average_Temp = (Min_Temp + Max_Temp)/2.**
4. Identify the day with the highest average temperature.
5. Plot a line graph of average temperature over time.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt


data = {
   'Date': ['2025-05-10','2025-05-11','2025-05-12','2025-05-13','2025-05-14','2025-05-15'],
   'Min_Temp': [25,26,None,24,27,28],
   'Max_Temp': [35,36,34,None,38,40]
}
df = pd.DataFrame(data)


df['Min_Temp'] = df['Min_Temp'].fillna(df['Min_Temp'].mean())
df['Max_Temp'] = df['Max_Temp'].fillna(df['Max_Temp'].mean())
df['Average_Temp'] = (df['Min_Temp'] + df['Max_Temp']) / 2


hottest = df.loc[df['Average_Temp'].idxmax(), 'Date']
print(f"Date with highest average temperature: {hottest}")
```

```
plt.plot(df['Date'], df['Average_Temp'], marker='o', color='orange')
plt.xlabel('Date')
plt.ylabel('Average Temperature')
plt.title('Average Temperature Over Time')
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

Output:

Date with highest average temperature: 2025-05-15

Result:
The program calculates average temperatures, identifies the hottest day, and visualizes the temperature trend.

TEST CASE 3: Daily Traffic Volume Report

Aim:
Using Google Colab, perform an analysis on a dataset of daily traffic volume to answer the following.

Requirements:

- Google Colab
- Python 3.x
- Libraries: pandas, matplotlib

Algorithm:

1. Create a dataset with Date and Vehicle_Count.
2. Replace missing values with 0.
3. Calculate total and average vehicle count.
4. Identify the day with the highest vehicle count.
5. Plot a line graph showing daily traffic trends.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
   'Date': ['2025-04-10','2025-04-11','2025-04-12','2025-04-13','2025-04-14','2025-04-15'],
   'Vehicle_Count': [500,450,800,None,600,550]
}
df = pd.DataFrame(data)

df['Vehicle_Count'] = df['Vehicle_Count'].fillna(0)
total = df['Vehicle_Count'].sum()
avg = df['Vehicle_Count'].mean()
print(f"Total vehicles: {total}")
print(f"Average daily vehicles: {avg:.2f}")

peak = df.loc[df['Vehicle_Count'].idxmax(), 'Date']
```

```
print(f"Date with highest traffic: {peak}")


plt.plot(df['Date'], df['Vehicle_Count'], marker='o', color='red')
plt.xlabel('Date')
plt.ylabel('Vehicle Count')
plt.title('Daily Traffic Volume Trend')
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

Result:
The program calculates total and average traffic, identifies the busiest day, and visualizes the traffic trend.

TEST CASE 4: E-Commerce Product Ratings

Aim:
Using Google Colab, perform an analysis on a dataset of e-commerce product ratings to answer the following.

Requirements:

- Google Colab
- Python 3.x
- Libraries: pandas, matplotlib

Algorithm:

1. Create a dataset of products with customer ratings.
2. Remove missing rating entries.
3. Calculate the average rating.
4. Identify the top three highest-rated products.
5. Plot a bar chart of top five product ratings.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
   'Product_Name': ['Laptop','Headphones','Smartwatch','Camera','Phone','Monitor'],
   'Customer_Rating': [4.8,4.5,None,4.7,4.9,4.6]
}
df = pd.DataFrame(data)

df = df.dropna(subset=['Customer_Rating'])
avg = df['Customer_Rating'].mean()
print(f"Average Rating: {avg:.2f}")

top = df.sort_values('Customer_Rating', ascending=False).head(3)
print("Top 3 Products:")
print(top[['Product_Name','Customer_Rating']])

df.sort_values('Customer_Rating', ascending=False).head(5).plot(kind='bar', x='Product_Name',
y='Customer_Rating', color='teal')
plt.ylabel('Rating')
```

```
plt.title('Top 5 Product Ratings')
plt.show()
```

Output:

Average Rating: 4.70

Top 3 Products:

	Product_Name	Customer_Rating
4	Phone	4.9
0	Laptop	4.8
3	Camera	4.7

Result:
The program calculates average ratings, identifies the top-rated products, and visualizes the ratings.

TEST CASE 5: Student Course Completion Report

Aim:
Using Google Colab, perform an analysis on a dataset of student course completion status to answer the following.

Requirements:

- Google Colab
- Python 3.x
- Libraries: pandas, matplotlib

Algorithm:

1. Create a dataset of students with course completion status.
2. Fill missing values in *Completion_Status* with “No.”
3. Count completed and not completed students.
4. Visualize completion versus non-completion using a pie chart.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
   'Student_ID': ['S001','S002','S003','S004','S005','S006'],
   'Completion_Status': ['Yes','No',None,'Yes','No','Yes']
}
df = pd.DataFrame(data)

df['Completion_Status'] = df['Completion_Status'].fillna('No')
count = df['Completion_Status'].value_counts()
print(count)

count.plot(kind='pie', autopct='%1.1f%%', colors=['lightgreen','lightcoral'], figsize=(6,6))
plt.title('Course Completion Status')
plt.ylabel('')
plt.show()
```

Output:

```
Yes    3
No     3
Name: Completion_Status, dtype: int64
```

(Pie chart displayed in Colab)

Result:
The program counts and visualizes student course completion versus non-completion status.